ARCHITECTURE BLUEPRINT

Lagos Rent Platform

Production-grade microservices architecture for verified listings, transparent pricing, auditable rentals, safe payments, and deliberate Spring Cloud and DevOps learning.

Spring Cloud Platform

Config Server, Eureka and API Gateway are first-class learning components.

Distributed Business Workflows

RabbitMQ, transactional outbox, idempotent consumers and a Rental-owned saga.

Operable by Design

Keycloak, database-per-service, OpenTelemetry, Docker, CI/CD and Kubernetes later.

Architecture baseline
Version 1.0

Prepared for a personal learning project
15 August 2026

01 - EXECUTIVE SUMMARY

Architecture in one page

The platform is a Lagos-first rental marketplace whose differentiator is trust: verified authority to list, explicit fees, admin review, immutable price snapshots, backend-verified payments, and auditable operations.

Business architecture

- Tenants discover approved listings and see the total move-in cost before applying.
- Landlords create properties and rental units, then submit listings for review.
- Admins verify accounts, approve listings and fees, monitor workflows, and resolve reports.
- Rental and payment state are coordinated through an explicit saga rather than a distributed transaction.

Technical architecture

- Nine Spring Boot deployables: Config Server, Eureka, Gateway, and six domain services.
- Keycloak for identity and OAuth2/OIDC; RabbitMQ for events and workflow commands.
- Database-per-service with PostgreSQL; MinIO/S3 for public media and private documents.
- Outbox/inbox reliability, OpenTelemetry observability, Docker Compose first, Kubernetes later.

> **Primary architectural choice:** Config Server and Eureka remain in the application because the project is deliberately optimized for learning Spring Cloud and DevOps. They are not accidental dependencies; their behavior, failure modes, and overlap with Kubernetes are part of the curriculum.

Recommended deployables

`config-server` `discovery-server` `api-gateway` `account-service` `listing-service` `search-service` `rental-service` `payment-service` `notification-service`

Definition of architectural success

The system is successful when each service is independently deployable, owns its database, publishes reliable versioned events, tolerates duplicate delivery, exposes observable workflow state, recovers after a dependency outage, and completes the rental-payment journey without shared tables or distributed two-phase commit.

DOCUMENT MAP

Contents

01 Executive summary
02 Architecture drivers
03 Product scope and trust model
04 System context
05 Target container architecture
06 Service catalog and ownership
07 Config Server and Eureka
08 Security architecture
09 Data architecture
10 Communication model
11 Event contracts and reliability
12 Listing publication flow
13 Rental-payment saga
14 Domain invariants
15 Admin architecture
16 Media, documents and privacy
17 Resilience and failure handling
18 Observability
19 Local deployment
20 CI/CD architecture
21 Kubernetes architecture
22 Repository strategy
23 Implementation sequence
24 Guardrails and definition of done
25 Risks and tradeoffs
26 ADR index
27 References

02 - ARCHITECTURE DRIVERS

Why the system is shaped this way

1

Trust

Verification status, document review, clear badges, reporting, and admin auditability are part of the core domain.

2

Price integrity

Fee lines and total move-in cost are backend-calculated and versioned. Rental applications capture immutable price snapshots.

3

Learning depth

The architecture intentionally includes Spring Cloud Config, Eureka, messaging, observability, CI/CD, and orchestration.

4

Independent ownership

Every service owns its data and state transitions. No service reaches into another database or repository.

5

Recoverability

Outbox/inbox, DLQs, reconciliation, saga compensation, and explicit manual review prevent silent corruption.

6

Operability

Traces, metrics, structured logs, health probes, deployment evidence, and runbooks are deliverables, not afterthoughts.

Quality attribute priorities

Attribute	Architectural response	Verification method
Security	Keycloak, gateway plus service token validation, private object storage, least privilege, audited admin actions	Authorization tests, threat model, secret scans, access-log review
Consistency	Single-service transactions, outbox/inbox, aggregate versions, saga state, immutable snapshots	Duplicate/out-of-order tests, crash tests, concurrency tests
Availability	Timeouts, circuit breakers, cached Eureka registry, durable queues, graceful shutdown, rolling deployment	Dependency outage and restart exercises
Scalability	Independent services, read projection, stateless application instances, asynchronous processing	Load tests and queue/backlog metrics
Maintainability	Bounded contexts, versioned contracts, ADRs, monorepo with independent builds, no shared domain library	Architecture review and module dependency checks
Observability	OpenTelemetry context across HTTP and AMQP, structured logs, dashboards, trace-linked saga state	Trace a complete listing and payment journey

03 - PRODUCT SCOPE

Trust-first, Lagos-first, expansion-ready

The first release focuses on one geography and one provider while keeping state, location, and provider abstractions explicit enough for later expansion.

Tenant promise

Real listings, clear prices, explicit fees, visible verification levels, protected payment, and no surprise charges after application.

Landlord promise

A direct route to serious tenants, structured listing management, inspection/application workflows, and reliable payment records.

Platform promise

Review listings, enforce transparent pricing, record authoritative payment state, support reports, and maintain an audit trail.

Admin promise

Resolve verification and listing queues, suspend suspicious entities, inspect saga/payment state, and justify every privileged action.

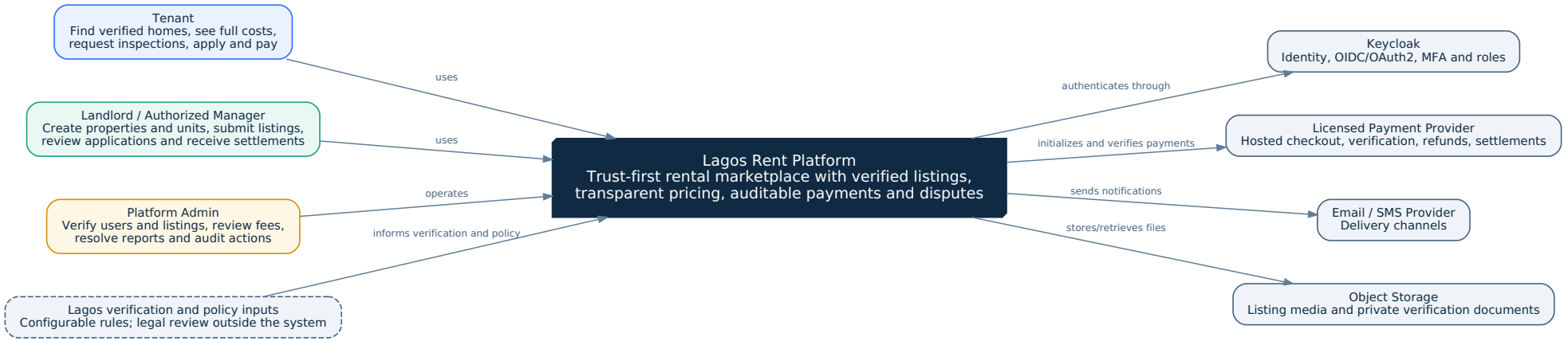
Initial functional scope

Tenant	Landlord	Admin
Profile, search, listing details, fee breakdown, saved listings, inspection request, application, payment, receipt, report	Profile and verification, property/unit creation, media, fee lines, review submission, inspection decisions, application decisions, payment status	Verification queues, listing review, user suspension, report/dispute handling, payment review, refunds, metrics, audit logs

Scope discipline: no multi-state rollout, active multi-provider payment strategy, internal wallet, native app, advanced chat, AI valuation, service mesh, or OpenSearch in the first complete workflow.

04 - C4 SYSTEM CONTEXT

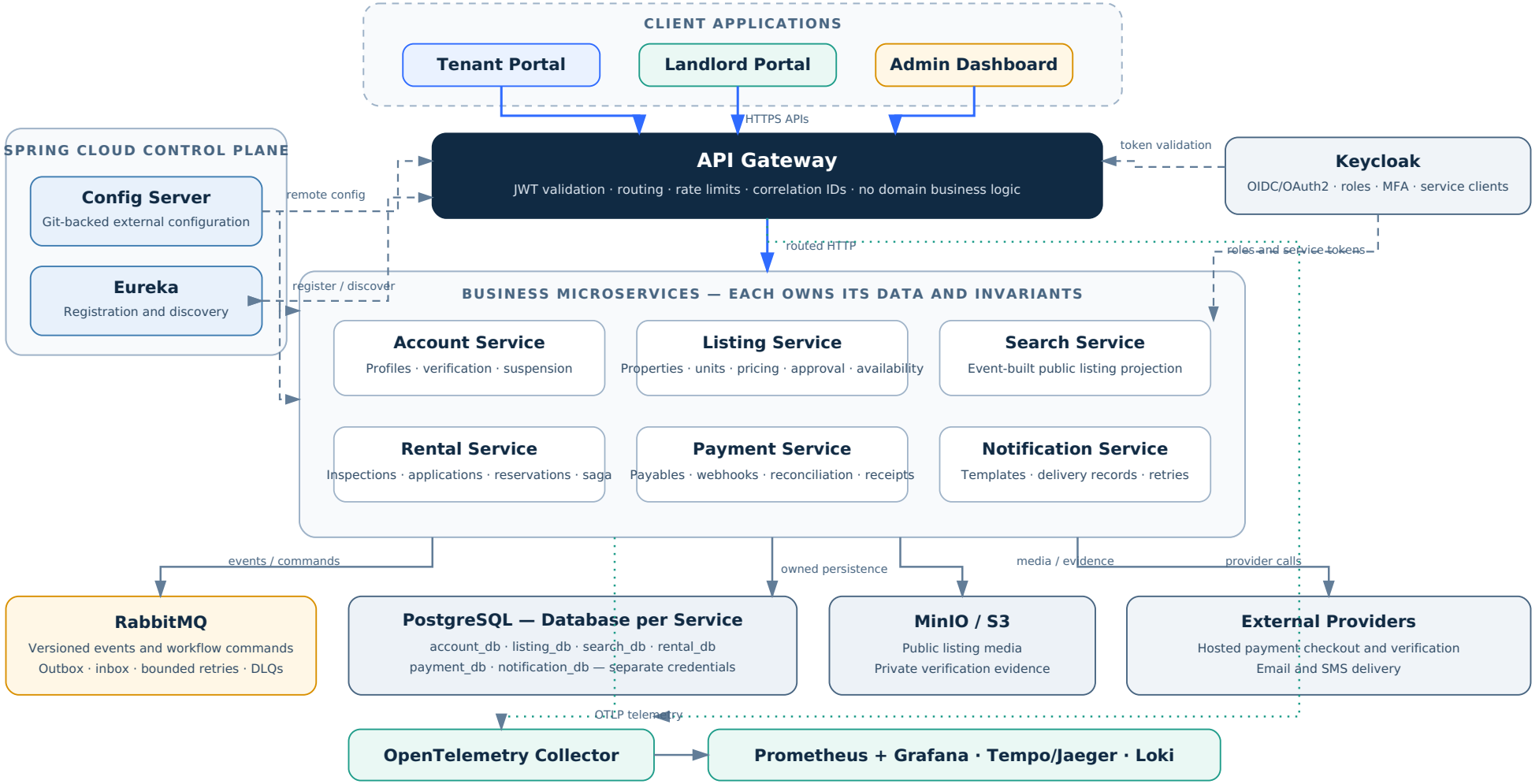
People and external systems



The platform sits between tenants, landlords/authorized managers, admins, identity, payment, communications, object storage, and configurable Lagos-specific verification/policy inputs.

05 - TARGET CONTAINER ARCHITECTURE

Runtime components and primary connections



Solid lines are primary application/data flows, dashed lines show configuration/discovery relationships, and dotted lines show telemetry export.

06 - SERVICE CATALOG

Bounded contexts and authority

AC Account Service

Owns: tenant/landlord profiles, verification cases, account suspension, preferences, saved listings, settlement references.

Key events: `landlord.verified.v1`, `account.suspended.v1`.

Boundary: Keycloak owns credentials, sessions, tokens and MFA.

LS Listing Service

Owns: properties, rental units, listings, fee lines, media metadata, review, publication, availability and atomic reservation.

Key events: `listing.published.v1`, `listing.reserved.v1`, `listing.rented.v1`.

Boundary: it does not own tenant applications or provider payment state.

SS Search Service

Owns: public listing-card projection, filters, pagination, search ranking and location taxonomy.

Input: listing lifecycle events.

Boundary: projection data is never authoritative for availability or price acceptance.

RS Rental Service

Owns: inspections, applications, decisions, price snapshots, reservations, tenancy and saga state.

Role: orchestrates the reservation-payment-tenancy workflow.

Boundary: Listing decides availability; Payment decides payment truth.

PS Payment Service

Owns: payables, payment attempts, provider webhooks, verification, refunds, settlements, reconciliation and receipts.

Rule: redirect success is never accepted as payment proof.

Boundary: stores immutable references/ snapshots, not foreign domain aggregates.

NS Notification Service

Owns: templates, delivery records, provider attempts, retries and dead-letter handling.

Input: domain events such as listing approval, application acceptance and payment success.

Rule: delivery failure never rolls back the originating business transaction.

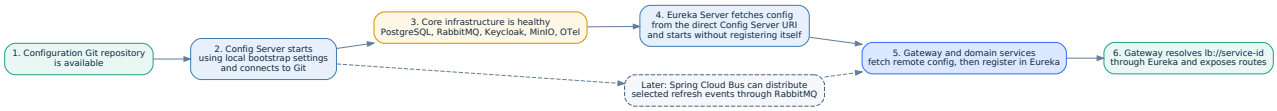
Platform services

Component	Responsibilities	Explicit non-responsibilities
Config Server	Serve versioned non-secret external configuration from Git	Secret store, business rules engine, service registry
Eureka	Register and discover Spring application instances	Discover databases, brokers, Keycloak, storage or external providers
API Gateway	Routing, token validation, rate limits, CORS, correlation, edge policy	Domain validation, orchestration, aggregate ownership
Keycloak	Identity, OAuth2/OIDC, credentials, sessions, roles, service clients	Landlord verification, listing approval, account business profile

07 - SPRING CLOUD PLATFORM

Config Server and Eureka without a startup cycle

Use config-first bootstrap: every service knows the Config Server's bootstrap address directly, then registers with Eureka after its environment is built.



The Config Server itself uses local bootstrap configuration for its Git backend. Discovery-first lookup can be explored later, after the direct flow is stable.

Minimal client bootstrap

```
spring:
  application:
    name: listing-service
  config:
    import: configserver:$
{CONFIG_SERVER_URL:http://localhost:8888}
```

Mandatory remote configuration should fail startup clearly. The optional import form is reserved for isolated tests or a deliberate standalone developer mode.

Single-node Eureka server

```
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

Gateway and domain services register. Config Server may register after startup for visibility, but clients initially locate it directly.

Configuration repository policy

Store in Git	Inject at runtime
Ports, timeouts, feature flags, queue names, upload limits, reservation duration, log levels, observability sampling	Database passwords, Keycloak client secrets, payment keys, SMTP credentials, RabbitMQ credentials, cloud credentials, signing keys

Spring Cloud Bus: RabbitMQ may later distribute selected refresh events. Structural settings and secrets should normally change through controlled restarts or rolling deployments rather than arbitrary runtime refresh.

08 - SECURITY ARCHITECTURE

Identity at the edge and authorization in every service

End-user flow

```
Browser
-> Keycloak Authorization Code flow with PKCE
-> access token
-> API Gateway
-> owning domain service
-> independent JWT validation and authorization
```

Internal service flow

```
Rental Service
-> Keycloak Client Credentials token
-> Listing Service internal API
-> audience/scope validation
-> authoritative response
```

Roles

TENANT LANDLORD ADMIN SUPER_ADMIN

Roles are identity claims; domain ownership and verification are still checked in the owning service.

Token validation

Validate issuer, signature, expiry, audience, scopes and roles in Gateway and downstream services.

Header hygiene

Strip externally supplied identity/internal headers such as `X-User-Id` and `X-Internal-Service`.

Authorization examples

Action	Required checks
Create listing	LANDLORD role, active Account, verified landlord state, ownership of Property/RentalUnit
Approve listing	ADMIN scope, review still open, optimistic concurrency, required reason/audit record
Apply for listing	TENANT role, active account, listing published, no conflicting active application
Initialize payment	Tenant owns accepted application, reservation confirmed, payable active, amount from server snapshot
Refund	Privileged payment scope, idempotency key, review state, reason, immutable audit record

09 - DATA ARCHITECTURE

Database-per-service and explicit data movement

Service	Database	Core aggregates	Important constraints
Account	`account_db`	Account, profiles, VerificationCase, SavedListing	Unique Keycloak subject; audited verification transitions
Listing	`listing_db`	Property, RentalUnit, Listing, Fee, Review, Reservation	Atomic reserve condition; listing version increases on material change
Search	`search_db`	ListingProjection, LocationTaxonomy	Unique listing ID; last aggregate version prevents stale updates
Rental	`rental_db`	Inspection, Application, PriceSnapshot, Reservation, Tenancy, Saga	Immutable accepted price snapshot; one active reservation path per application
Payment	`payment_db`	Payable, Attempt, Transaction, Webhook, Refund, Receipt	Unique provider/internal references; idempotent webhook processing
Notification	`notification_db`	Template, Delivery, ProviderAttempt	Unique event/template/recipient key prevents duplicates

Forbidden: cross-service foreign keys, shared repositories, reporting joins across operational databases, and direct admin database edits.

Read-model strategy

Search consumes listing events and builds a public projection. PostgreSQL is sufficient for the first version; OpenSearch can be introduced when ranking, typo tolerance, scale or operational independence justify it.

Object-storage strategy

Bucket/class	Content	Access model
Public listing media	Approved listing photos and optimized derivatives	CDN/public read after moderation; upload through signed URL
Private verification documents	Identity, authority and property documents	Private; short-lived signed URLs; access logged
Private receipts/ agreements	Generated or uploaded tenant documents	Authorized parties only; retention policy

10 - COMMUNICATION MODEL

HTTP for immediate authority, RabbitMQ for durable coordination

Synchronous HTTP

Use when the caller requires an immediate authoritative answer or a user-facing command result.

- Gateway to owning service
- Rental obtains current listing/price data before snapshot
- Payment verifies transaction with provider
- Admin executes a privileged command on the owner

Required: timeout, authentication, correlation, explicit failure mapping.

Asynchronous RabbitMQ

Use for completed domain facts, durable workflow commands, projections, notifications and compensation.

- Listing lifecycle events to Search
- Reservation commands/events across Rental and Listing
- Payment results to Rental
- Business events to Notification

Required: versioned schema, outbox, publisher confirms, idempotent consumer, DLQ.

API conventions

```
Public:   /api/v1/{domain}/...
Admin:    /api/v1/admin/{domain}/...
Internal: /internal/v1/{domain}/...

Every response/error includes a traceId.
Mutating idempotent operations accept an Idempotency-Key where applicable.
```

11 - EVENT RELIABILITY

Versioned contracts, transactional outbox, idempotent inbox

Common event envelope

```
{
  "eventId": "34d86bc5-3945-41ce-8d51-0ba9460db558",
  "eventType": "listing.published.v1",
  "aggregateType": "listing",
  "aggregateId": "fa3f3a60-5a84-4514-8731-12db97eff937",
  "aggregateVersion": 8,
  "occurredAt": "2026-08-13T16:30:00Z",
  "producer": "listing-service",
  "correlationId": "b07a9bf1-6e49-45cb-a15f-6eb9da83247b",
  "causationId": "75082d46-d726-4365-a063-357c7d30410c",
  "payload": {}
}
```

Outbox publication

```
DB transaction:
  update aggregate
  insert outbox event
  commit

Publisher:
  claim rows
  publish with confirm
  mark published
```

Start with polling. Debezium CDC is a later learning extension.

Inbox consumption

```
DB transaction:
  if eventId processed: no-op
  else apply state change
  insert processed_message
  commit
  acknowledge
```

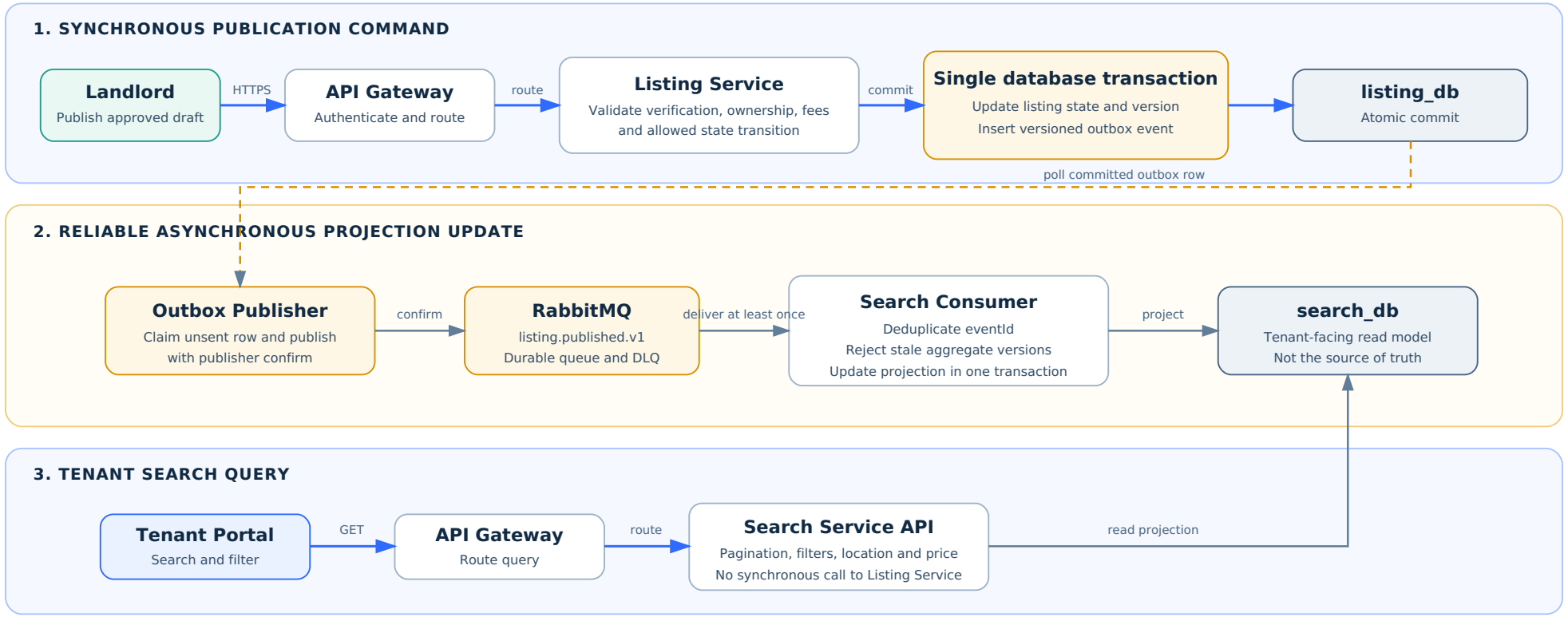
Assume at-least-once delivery and safe repetition, never exactly-once delivery.

RabbitMQ topology

Element	Convention	Operational rule
Exchange	`platform.domain.v1`	Durable; routing by versioned domain key
Routing key	`<domain>.<event-name>.v1`	Fact or explicit workflow command
Queue	`<consumer>.<purpose>.v1`	One logical responsibility per queue
Retry	Bounded delay queues	No infinite requeue loop
DLQ	`<queue>.dlq`	Alert, inspect, replay through controlled tooling

12 - FIRST DISTRIBUTED VERTICAL SLICE

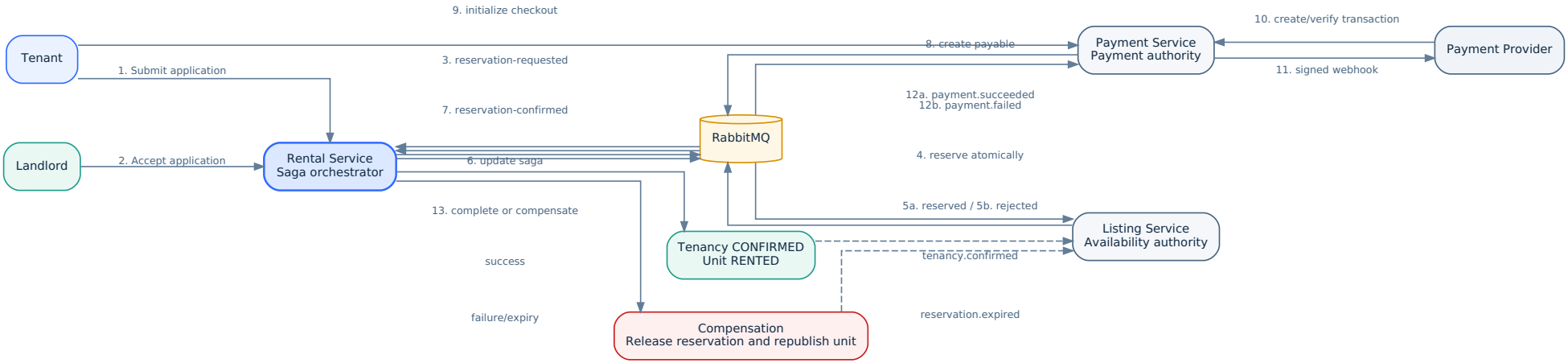
Listing publication to search projection



The first meaningful milestone proves authentication, discovery, gateway routing, database ownership, outbox publishing, RabbitMQ delivery, idempotent projection and tenant search.

13 - DISTRIBUTED WORKFLOW

Rental-payment saga and compensation



Rental Service orchestrates workflow state; Listing Service owns availability; Payment Service owns provider truth. Failures trigger explicit compensation or manual review.

14 - DOMAIN INVARIANTS

Rules that protect trust and price integrity

1. A listing cannot be submitted without a complete backend-calculated price breakdown.
2. A listing cannot be published without landlord verification and admin approval.
3. Every fee records type, amount, recipient, frequency, refundability and explanation.
4. An `OTHER` fee requires explicit admin approval.
5. The exact property address remains private until the approved workflow stage.
6. A rental application stores an immutable listing version and price snapshot.
7. Only Listing Service can atomically reserve or release a unit.
8. No rent payment begins before an accepted application and confirmed reservation.
9. Payment amount and currency must match the server-side payable.
10. Provider redirects do not mark payments successful; signed webhook plus backend verification is required.
11. Duplicate webhooks and events produce one effective transition.
12. Late payment after reservation expiry enters explicit review; it is not silently accepted or ignored.
13. Every privileged admin mutation requires a reason and immutable audit record.

Core state machines

Listing

`DRAFT` -> `SUBMITTED_FOR_REVIEW` -> `APPROVED` -> `PUBLISHED` -> `RESERVED` -> `RENTED`

Alternatives: changes requested, rejected, suspended, reservation released back to published.

Application and reservation

`SUBMITTED` -> `UNDER_REVIEW` -> `ACCEPTED` -> `RESERVATION_REQUESTED` -> `CONFIRMED` -> `PAID / EXPIRED`

Payment

`CREATED` -> `PENDING` -> `SUCCEEDED / FAILED / EXPIRED / REVIEW_REQUIRED` -> `REFUNDED`

15 - ADMIN ARCHITECTURE

A privileged client, not a catch-all database service

The Admin Dashboard routes each operation to the service that owns the state. This preserves boundaries and prevents a central admin service from becoming a shared-database backdoor.

Admin capability	Owning API	Required audit detail
Landlord verification, account suspension	Account Service	Actor, decision, reason, prior/new status, document references
Listing review, fee approval, suspension	Listing Service	Listing version, fee changes, reason, reviewer, timestamps
Inspection/application/reservation review	Rental Service	Saga/application state, actor, reason, correlation/trace IDs
Payment verification, refund, reconciliation	Payment Service	Provider refs, idempotency key, amount/currency, evidence, actor
Delivery failures and replay	Notification Service	Event ID, template, provider attempt, replay actor

Cross-service dashboards

For the MVP, the UI may compose a small number of owner API calls. When reporting volume grows, introduce an event-built Analytics/Reporting Service rather than querying every operational database.

> **Administrative safety:** destructive actions require role/scope checks, a reason, optimistic concurrency, and an immutable audit record. Sensitive exports require separate authorization and PII review.

16 - PRIVACY AND DOCUMENT SECURITY

Separate public listing data from sensitive evidence

Public listing surface

- Area, LGA, approximate location and landmark
- Approved photos and amenities
- Rent, fee lines, total move-in cost
- Verification badges with clear meaning

Private surface

- Exact address before approved inspection stage
- Identity and authority documents
- Bank/settlement references and provider metadata
- Complaint evidence and sensitive audit context

Controls

- Private buckets and short-lived signed URLs.
- Malware/content scanning extension point for uploads.
- Access logs for sensitive documents.
- PII minimization in events, logs and traces.
- Retention and deletion policies by data class.
- Encryption in transit and at rest; tightly scoped service credentials.

> Legal and regulatory rules should be represented as configurable policy and reviewed by qualified professionals before public launch. The architecture preserves the records needed for receipts, decisions, disputes and audit without claiming to replace legal review.

17 - RESILIENCE

Failure is a normal operating mode

Failure	Expected behavior	Mechanism
Config Server unavailable during new startup	Mandatory clients fail clearly; existing healthy instances continue	Config-first bootstrap, probes, controlled rolling changes
Eureka temporarily unavailable	Clients use cached registry while attempting reconnection	Client registry cache, sensible timeouts, multiple peers later
RabbitMQ unavailable during commit	Business transaction commits; event remains in outbox	Transactional outbox and retrying publisher
Consumer receives duplicate	One effective state change	Inbox/processed message key and idempotent handler
Events arrive out of order	Older aggregate version ignored or reconciled	Aggregate version in envelope/ projection
Payment webhook duplicated	One transaction transition and receipt	Unique provider reference and webhook idempotency
Payment succeeds after reservation expiry	Explicit review and reconciliation	Saga manual-review state and provider query
Notification provider unavailable	Business transaction remains complete; delivery retries/DLQ	Asynchronous Notification Service

Retry policy

Safe candidates: read-only GET, provider verification query, idempotent command with an idempotency key, temporary connection failure before side effects.

Unsafe by default: create payment, refund, accept application, reserve listing, authorization or validation failures.

Operational recovery artifacts

Maintain runbooks for DLQ replay, stuck saga resolution, outbox backlog, pending payment reconciliation, failed configuration rollout, database restore and controlled service rollback/forward-fix.

18 - OBSERVABILITY

One trace across HTTP, RabbitMQ and provider calls

Telemetry pipeline

```
Spring services and Gateway
-> OpenTelemetry Java agent/starter
-> OTLP
-> OpenTelemetry Collector
-> Prometheus/Grafana (metrics)
-> Tempo or Jaeger (traces)
-> Loki or log platform (logs)
```

Logs

service, environment, traceId, spanId, correlationId, route, status, duration, event type and permitted actor context.

Metrics

latency, errors, queue depth, DLQ count, outbox age, projection lag, saga duration, pending-payment age, webhook failures.

Traces

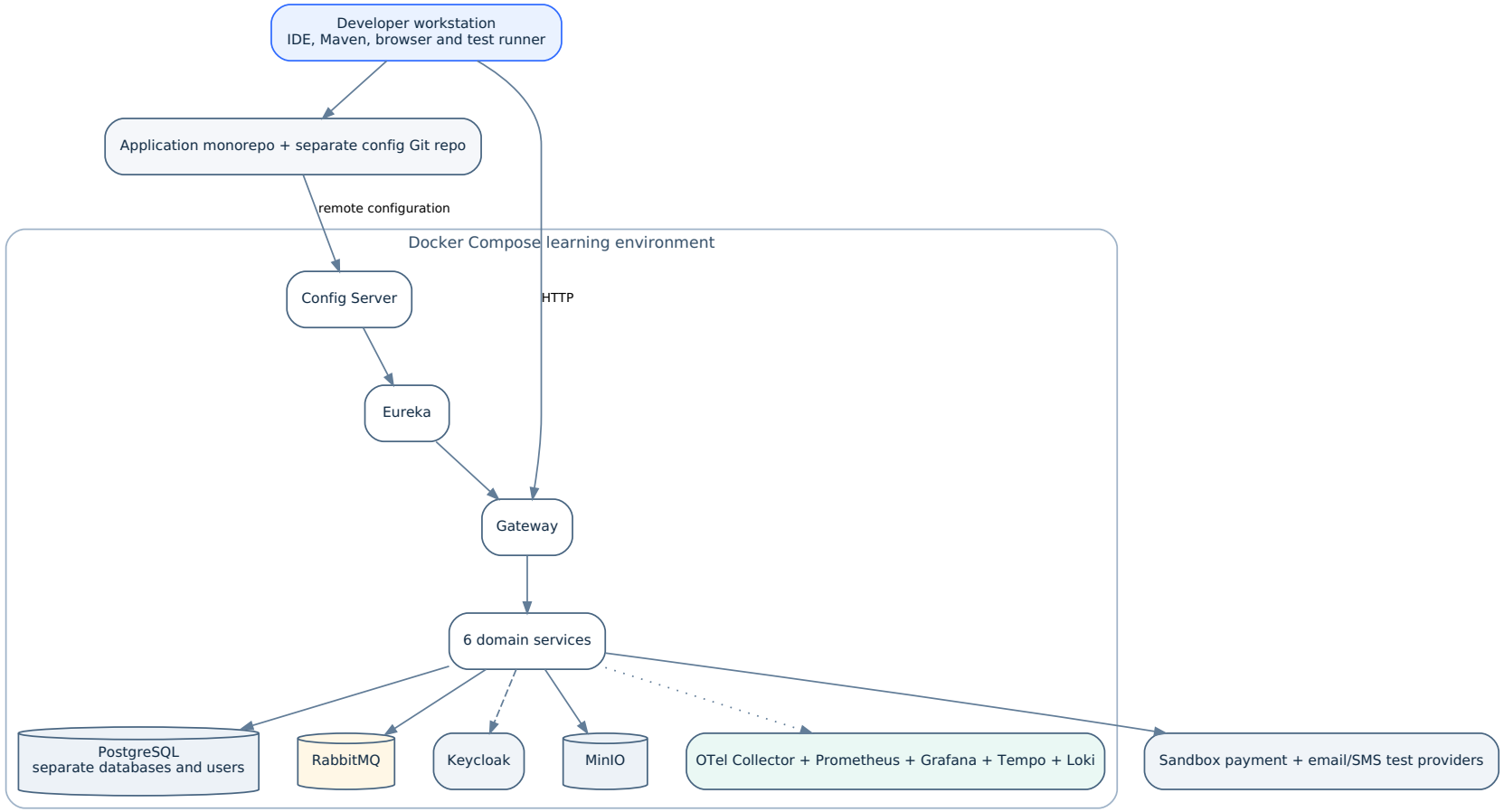
Propagate W3C context through Gateway, HTTP clients, AMQP headers, consumers and payment verification calls.

Priority dashboards and alerts

Dashboard/alert	Why it matters
Listing event time vs Search projection update time	Measures actual eventual-consistency lag
Oldest unpublished outbox event	Detects broken event publication before data drifts
DLQ growth and consumer failures	Shows poison messages and contract/processing failures
Stuck saga and reservation age	Prevents units remaining locked indefinitely
Pending payment and reconciliation age	Detects provider/webhook disagreement and customer-impacting uncertainty
Eureka registrations and Config request failures	Shows platform control-plane degradation

19 - LOCAL DEPLOYMENT VIEW

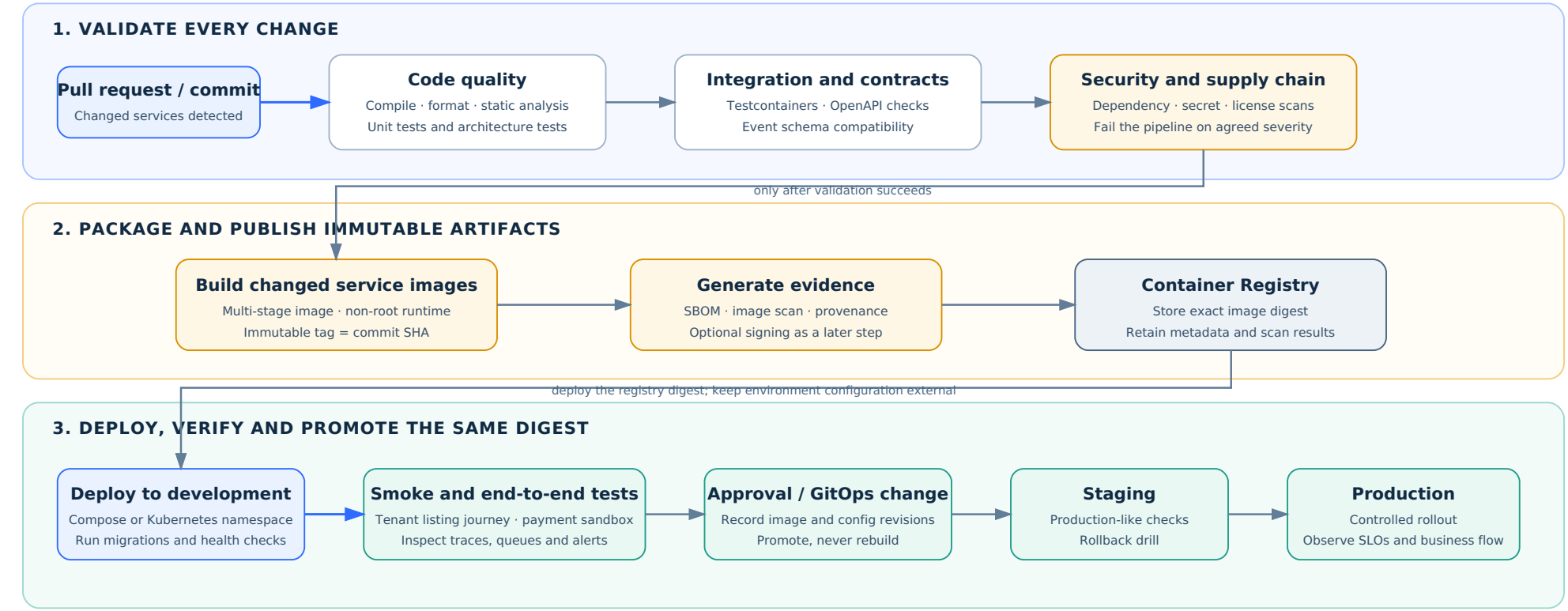
Docker Compose learning environment



One command should start a repeatable environment with health checks, named volumes, separate databases/users, sandbox providers and the full observability path.

20 - DELIVERY ARCHITECTURE

CI/CD pipeline and promotion flow



Build immutable images, scan them, deploy exact digests, execute smoke/end-to-end tests, and promote the same artifact through environments.

21 - KUBERNETES ARCHITECTURE

Orchestration after the end-to-end workflow works

Cluster components

Ingress / Gateway API | API Gateway | Config Server | Eureka | Domain Deployments | Kubernetes Services | OTel Collector | Prometheus/Grafana | Secrets | NetworkPolicies

Two intentional discovery profiles

Profile	Behavior	Learning objective
`kubernetes-eureka`	Applications register with Eureka; Gateway resolves `lb://service-id`; Kubernetes Services still provide stable networking	Operate the Spring Cloud design in a cluster and observe overlap/failure behavior
`kubernetes-native`	Eureka clients disabled; applications use Kubernetes Services/DNS	Compare application-level registry discovery with platform-native discovery

Deployment requirements

- Readiness, liveness and startup probes; graceful shutdown and rolling-update tests.
- Resource requests/limits, NetworkPolicies, PodDisruptionBudgets and autoscaling only where justified.
- Helm or Kustomize with environment-specific values; immutable image digests.
- Managed PostgreSQL/object storage and a supported RabbitMQ deployment for realistic production-like environments.
- GitOps with Argo CD or Flux as a later DevOps learning phase.

22 - REPOSITORY STRATEGY

Monorepo for application code, separate repo for configuration

Application monorepo

```
lagos-rent-platform/
├── services/
│   ├── config-server/
│   ├── discovery-server/
│   ├── api-gateway/
│   ├── account-service/
│   ├── listing-service/
│   ├── search-service/
│   ├── rental-service/
│   ├── payment-service/
│   └── notification-service/
├── frontend/web-application/
├── contracts/{openapi,asyncapi,event-
schemas}/
├── platform/{docker-
compose,keycloak,rabbitmq,observability,kubern
etes,helm}/
├── docs/{architecture,adr,diagrams,runbooks}/
├── .github/workflows/
└── pom.xml
```

Configuration repo

```
lagos-rent-config/
├── application.yml
├── application-local.yml
├── application-docker.yml
├── application-staging.yml
├── application-production.yml
├── discovery-server.yml
├── api-gateway.yml
├── account-service.yml
├── listing-service.yml
├── search-service.yml
├── rental-service.yml
├── payment-service.yml
└── notification-service.yml
```

No shared domain library: the root Maven build may manage versions and quality plugins, but must not distribute JPA entities, domain DTOs, repositories, validation rules or payment/listing models.

23 - IMPLEMENTATION SEQUENCE

Build complete vertical slices in dependency order

Order	Phase	Required proof before continuing
1	Architecture, ADRs, state machines and service ownership	Every state and transition has one owner
2	Monorepo, compatible BOM, CI build	Clean checkout passes `mvn verify`
3	Configuration repo and Config Server	Remote profile/property retrieval and clear failure behavior
4	Eureka and bootstrap sequence	Sample service registers/discovers without circular dependency
5	Docker Compose platform, Keycloak and Gateway	Authenticated Gateway route to discovered health endpoint
6	Service template and Account Service	Profiles/verification with security, migrations and tests
7	Listing Service and transparent pricing	Admin-approved listing with immutable versioned fee data
8	RabbitMQ contracts, outbox/inbox and Search	Listing projection recovers from broker/Search outages
9	Rental Service and price snapshots	Inspection/application flow with immutable snapshot
10	Payment Service and provider sandbox	Signed webhook plus backend verification; duplicate safe
11	Rental-payment saga and compensation	No double reservation; failure/expiry releases unit
12	Notification and admin operations	Async retries/DLQ and fully audited privileged actions
13	Resilience, observability and complete test suite	Failure scenarios recover without manual data repair
14	CI/CD, registry, deployment evidence	Exact commit/image/config revision is traceable
15	Kubernetes, Helm/GitOps comparison and hardening	Rolling deployment preserves the complete user journey

First milestone: landlord publishes a minimal listing, Listing writes an outbox event, RabbitMQ delivers it, Search stores an independent projection, and a tenant retrieves it through the Gateway with one distributed trace.

24 - GUARDRAILS AND DEFINITION OF DONE

What makes this a real distributed system

Architectural guardrails

1. No shared database or JPA entity.
2. No business logic in Gateway.
3. No distributed transaction manager.
4. Every event is versioned and published through outbox.
5. Every consumer is idempotent.
6. Every synchronous dependency has a timeout.
7. Search remains a projection.
8. Payment proof comes from backend verification.
9. Admin mutations are audited.
10. Secrets remain outside config Git.

Distributed MVP definition of done

- Services independently build and deploy.
- Broker outage does not lose committed events.
- Duplicate and out-of-order events are tolerated.
- Reservation uses an observable saga.
- Compensation releases failed reservations.
- One trace crosses HTTP and RabbitMQ boundaries.
- Stuck sagas, DLQs, outbox backlog and pending payments are visible.
- One service can stop and later recover.
- Payment webhooks are idempotent and reconciled.
- Backups, restore and incident runbooks exist.

Mandatory failure tests

RabbitMQ outage | Search offline | duplicate event | out-of-order event | consumer crash after commit | publisher crash after send | duplicate webhook | late webhook | notification provider failure | Config/Eureka restart

25 - RISKS AND TRADEOFFS

Deliberate complexity with explicit mitigations

Risk/tradeoff	Impact	Mitigation
Nine Spring applications plus infrastructure	High local and cognitive overhead	Vertical slices, monorepo, standard templates, Compose profiles, strong documentation
Config Server/Eureka startup dependency	Control-plane outage can block new instances	Direct config bootstrap, health checks, cached registry, peer Eureka later, controlled rollout
Eventual consistency	Search/admin screens can lag	Projection lag metrics, user-visible states, rebuild tooling, aggregate versions
Saga complexity	Partial failure and ambiguous states	Persisted saga steps, compensation, deadlines, manual review, trace-linked admin view
Payment/regulatory exposure	Trust, compliance and financial loss risk	Licensed provider, hosted checkout, no self-operated wallet/escrow, reconciliation and legal review
Sensitive documents	Privacy and fraud risk	Private storage, signed URLs, access logs, least privilege, retention rules
Eureka overlaps with Kubernetes discovery	Duplicate mechanisms and operational cost	Treat as a documented learning profile; compare against Kubernetes-native discovery later

26 - ARCHITECTURE DECISION RECORDS

Recommended ADR index

1. Lagos-only MVP
2. Transparent fee lines and total move-in cost
3. Keycloak as identity provider
4. Initial microservice boundaries
5. Database-per-service
6. RabbitMQ for events and workflow commands
7. Transactional outbox
8. Inbox/idempotent consumers
9. Rental Service as saga orchestrator

1. Search as event-built projection
2. Spring Cloud Config Server
3. Eureka service discovery
4. Config-first bootstrap
5. Secrets outside config Git
6. Admin calls owning service APIs
7. Docker Compose before Kubernetes
8. OpenTelemetry observability
9. One payment provider behind an adapter

Each ADR should contain context, decision, alternatives, consequences, status and a review trigger.

27 - REFERENCES

Official technical references

1. **Spring Cloud release train compatibility.** spring.io/projects/spring-cloud
2. **Spring Cloud Config Client and Config Data import.** docs.spring.io/spring-cloud-config/reference/client.html
3. **Spring Cloud Netflix / Eureka.** docs.spring.io/spring-cloud-netflix/reference/spring-cloud-netflix.html
4. **Spring Cloud Bus.** docs.spring.io/spring-cloud-bus/reference/quickstart.html
5. **Keycloak OpenID Connect.** keycloak.org/securing-apps/oidc-layers
6. **RabbitMQ reliability and confirms.** rabbitmq.com/docs/reliability and rabbitmq.com/docs/confirms
7. **OpenTelemetry Java instrumentation.** opentelemetry.io/docs/languages/java/instrumentation
8. **Kubernetes Services and DNS.** kubernetes.io/docs/concepts/services-networking/service and kubernetes.io/docs/concepts/services-networking/dns-pod-service

Version baseline

At the preparation date, the official Spring compatibility table mapped Spring Cloud 2025.1.x to Spring Boot 4.0.x and 4.1.x beginning with the relevant service release. Pin exact compatible patch versions through the Spring Cloud BOM and re-check before upgrades.

> **Companion implementation guide:** the Markdown deliverable provides the complete phase-by-phase build order, checklists, exit criteria, test scenarios and deployment steps that operationalize this architecture.